

Structural Vibration Analyzer

Full ESP32 Firmware Documentation

Contents

1	Purpose of the ESP32 Firmware	6
2	Hardware Communication Overview	6
2.1	Why the PCA9548A Multiplexer Is Needed	6
2.2	Physical Wiring Assumptions	7
3	Imported Libraries	7
3.1	<code>#include <Arduino.h></code>	7
3.1.1	Library Origin	7
3.1.2	Purpose	7
3.1.3	How It Is Used in This Project	8
3.2	<code>#include <Wire.h></code>	8
3.2.1	Library Origin	8
3.2.2	Purpose	8
4	Constants and Global Settings	9
4.1	MPU6050 I2C Address	9
4.1.1	Purpose	9
4.2	PCA9548A I2C Address	9
4.2.1	Purpose	9
4.3	ESP32 I2C Pins	9
4.3.1	Purpose	10
4.4	Serial Baud Rate	10
4.4.1	Purpose	10
4.5	Sampling Rate	10
4.5.1	Purpose	10
4.6	Number of Sensors	11
4.6.1	Purpose	11
4.7	Sensor Channels	11
4.7.1	Purpose	11
4.8	Calibration Samples	11
4.8.1	Purpose	12

4.9	Accelerometer Scale Factor	12
4.9.1	Purpose	12
4.10	Gyroscope Scale Factor	12
4.10.1	Purpose	12
4.11	Binary Packet Constants	13
4.11.1	Purpose	13
5	Data Structures and Global Variables	14
5.1	<code>SensorOffset</code> Structure	14
5.1.1	Purpose	14
5.1.2	Why Floating-Point Values Are Used	14
5.2	<code>streamStartUs</code>	14
5.2.1	Purpose	15
5.3	<code>offsets</code>	15
5.3.1	Purpose	15
5.4	<code>sampleIndex</code>	15
5.4.1	Purpose	15
6	Function Breakdown	15
6.1	<code>selectPCAChannel()</code>	15
6.1.1	Function Origin	16
6.1.2	Purpose	16
6.1.3	Input	16
6.1.4	Output	16
6.1.5	Internal Library Functions Used	16
6.1.6	How It Is Used in the Program	17
6.2	<code>writeMPU()</code>	18
6.2.1	Function Origin	18
6.2.2	Purpose	18
6.2.3	Inputs	18
6.2.4	Output	18
6.2.5	Internal Library Functions Used	18
6.2.6	How It Is Used	19
6.3	<code>readMPU()</code>	19
6.3.1	Function Origin	19
6.3.2	Purpose	19
6.3.3	Inputs	20
6.3.4	Output	20
6.3.5	Internal Library Functions Used	20
6.3.6	Why 14 Bytes Are Requested	21
6.3.7	How the 16-bit Values Are Reconstructed	22
6.3.8	How It Is Used	22
6.4	<code>mpuPresentOnSelectedChannel()</code>	22
6.4.1	Function Origin	22
6.4.2	Purpose	22

6.4.3	Input	22
6.4.4	Output	23
6.4.5	Internal Library Functions Used	23
6.4.6	How It Is Used	23
6.5	setupMPUOnSelectedChannel()	23
6.5.1	Function Origin	24
6.5.2	Purpose	24
6.5.3	Input	24
6.5.4	Output	24
6.5.5	Internal Functions Used	24
6.5.6	Register Configuration	24
6.6	clampToInt16()	25
6.6.1	Function Origin	25
6.6.2	Purpose	25
6.6.3	Input	26
6.6.4	Output	26
6.6.5	Internal Library Function Used	26
6.6.6	Why This Function Is Needed	26
6.7	appendU8()	26
6.7.1	Function Origin	27
6.7.2	Purpose	27
6.7.3	Inputs	27
6.7.4	Output	27
6.7.5	How It Works	27
6.7.6	Library Functions Used	27
6.8	appendU32()	27
6.8.1	Function Origin	28
6.8.2	Purpose	28
6.8.3	Inputs	28
6.8.4	Output	28
6.8.5	Little-Endian Format	28
6.8.6	Bitwise Operations Used	28
6.8.7	Library Functions Used	29
6.9	appendI16()	29
6.9.1	Function Origin	29
6.9.2	Purpose	29
6.9.3	Inputs	29
6.9.4	Output	29
6.9.5	Little-Endian Format	29
6.9.6	Library Functions Used	29
6.10	calculateChecksum()	30
6.10.1	Function Origin	30
6.10.2	Purpose	30
6.10.3	Inputs	30
6.10.4	Output	30

6.10.5	Checksum Equation	30
6.10.6	How It Is Used	31
6.10.7	Library Functions Used	31
6.11	calibrateSensors()	31
6.11.1	Function Origin	31
6.11.2	Purpose	31
6.11.3	Inputs	31
6.11.4	Outputs	32
6.11.5	Internal Custom Functions Used	32
6.11.6	Internal Library Functions Used	32
6.11.7	Calibration Process	33
6.11.8	Offset Equations	33
6.11.9	Special Treatment of the Y-Axis	33
6.11.10	Important Assumption	34
6.12	sendBinaryFrame()	34
6.12.1	Function Origin	34
6.12.2	Purpose	34
6.12.3	Inputs	35
6.12.4	Output	35
6.12.5	Internal Custom Functions Used	35
6.12.6	Internal Library Functions Used	36
6.12.7	Timestamp Calculation	36
6.12.8	Sample Index	36
6.12.9	Sensor Reading and Offset Correction	36
6.12.10	Failure Handling	37
6.12.11	Binary Frame Layout	37
6.13	setup()	37
6.13.1	Function Origin	37
6.13.2	Purpose	37
6.13.3	Inputs	38
6.13.4	Outputs	38
6.13.5	Internal Library Functions Used	38
6.13.6	Internal Custom Functions Used	39
6.13.7	Setup Sequence	39
6.13.8	Importance of START_BINARY	40
6.14	loop()	40
6.14.1	Function Origin	40
6.14.2	Purpose	40
6.14.3	Inputs	41
6.14.4	Outputs	41
6.14.5	Internal Library Functions Used	41
6.14.6	Internal Custom Functions Used	41
6.14.7	Sampling Rate Logic	41
6.14.8	Why lastSample += SAMPLE_PERIOD_US Is Used	42

7	How the ESP32 Reads Sensor Data	42
8	How the ESP32 Outputs Data	43
8.1	Why Binary Output Is Used	43
8.2	Full Frame Size	43
9	Sampling Rate Implementation	44
10	Complete ESP32 Firmware Execution Flow	44
11	Important Design Decisions	45
11.1	Use of Binary Packets	45
11.2	Use of Sync Bytes	46
11.3	Use of Checksum	46
11.4	Use of Sample Index	46
11.5	Use of Timestamp	46
12	Summary	46

1 Purpose of the ESP32 Firmware

The purpose of the ESP32 firmware is to collect acceleration and gyroscope data from three MPU6050 sensors using a PCA9548A I2C multiplexer, package the data into a compact binary format, and transmit the data to a computer through high-speed serial communication.

The ESP32 firmware performs the following main tasks:

1. Initializes serial communication.
2. Initializes I2C communication.
3. Selects each MPU6050 sensor through the PCA9548A multiplexer.
4. Checks whether each MPU6050 sensor is present.
5. Configures each MPU6050 sensor.
6. Calibrates the accelerometer and gyroscope offsets.
7. Reads acceleration and gyroscope values from all three sensors.
8. Applies offset correction.
9. Packages the readings into a binary frame.
10. Sends the binary frame to the computer.
11. Controls the sampling rate using the ESP32 microsecond timer.

The ESP32 firmware acts as the real-time data acquisition layer of the Structural Vibration Analyzer.

The general data path is:

MPU6050 Sensors → PCA9548A I2C Multiplexer → ESP32 → Binary Serial Output → Python Logger

2 Hardware Communication Overview

2.1 Why the PCA9548A Multiplexer Is Needed

Each MPU6050 sensor normally uses the same I2C address:

`0x68`

If three MPU6050 sensors with the same address are connected directly to the same I2C bus, the ESP32 cannot distinguish between them. The PCA9548A solves this problem by separating the sensors into different I2C channels.

In this project:

- Sensor 1 is connected to PCA9548A channel 0.
- Sensor 2 is connected to PCA9548A channel 1.
- Sensor 3 is connected to PCA9548A channel 2.

The ESP32 selects one channel at a time. Once a channel is selected, only the MPU6050 on that channel communicates with the ESP32.

2.2 Physical Wiring Assumptions

The firmware comments describe the wiring as:

- ESP32 SDA → PCA9548A SDA
- ESP32 SCL → PCA9548A SCL
- ESP32 3.3 V → PCA9548A VIN/VCC and MPU6050 VCC
- ESP32 GND → common ground

The ESP32 uses:

SDA = *GPIO21*

SCL = *GPIO22*

3 Imported Libraries

3.1 #include <Arduino.h>

```
#include <Arduino.h>
```

3.1.1 Library Origin

`Arduino.h` is part of the Arduino framework. It is not custom-made for this project.

3.1.2 Purpose

This library provides the standard Arduino and ESP32 functions used by the firmware. Important functions from `Arduino.h` used in the script include:

- `setup()`
- `loop()`
- `Serial.begin()`

- `Serial.print()`
- `Serial.println()`
- `Serial.write()`
- `Serial.flush()`
- `delay()`
- `micros()`
- `lround()`

3.1.3 How It Is Used in This Project

The Arduino framework provides the basic runtime structure. The function `setup()` runs once when the ESP32 starts, and `loop()` runs continuously afterward.

The `Serial` object is used to send startup messages and binary sensor frames to the computer.

The `micros()` function is used to control the sampling rate.

3.2 `#include <Wire.h>`

```
#include <Wire.h>
```

3.2.1 Library Origin

`Wire.h` is the standard Arduino I2C communication library. It is not custom-made for this project.

3.2.2 Purpose

This library allows the ESP32 to communicate with I2C devices.

In this project, it is used to communicate with:

- The PCA9548A I2C multiplexer.
- The MPU6050 accelerometer/gyroscope sensors.

Important functions from `Wire.h` used in the script include:

- `Wire.begin()`
- `Wire.setClock()`
- `Wire.beginTransmission()`

- `Wire.write()`
- `Wire.endTransmission()`
- `Wire.requestFrom()`
- `Wire.read()`

4 Constants and Global Settings

4.1 MPU6050 I2C Address

```
#define MPU_ADDR 0x68
```

4.1.1 Purpose

This defines the I2C address of the MPU6050 sensors.

$$\text{MPU_ADDR} = 0x68$$

All three MPU6050 sensors use this same address. The PCA9548A allows them to be separated by channel.

4.2 PCA9548A I2C Address

```
#define PCA_ADDR 0x70
```

4.2.1 Purpose

This defines the I2C address of the PCA9548A multiplexer.

$$\text{PCA_ADDR} = 0x70$$

The ESP32 sends channel-selection commands to this address.

4.3 ESP32 I2C Pins

```
#define SDA_PIN 21  
#define SCL_PIN 22
```

4.3.1 Purpose

These define the ESP32 pins used for I2C communication.

$$SDA = GPIO21$$

$$SCL = GPIO22$$

SDA carries the I2C data signal. SCL carries the I2C clock signal.

4.4 Serial Baud Rate

```
const uint32_t SERIAL_BAUD = 921600;
```

4.4.1 Purpose

This defines the serial communication speed between the ESP32 and the computer.

$$\text{Baud rate} = 921600 \text{ bits/s}$$

A high baud rate is used because the ESP32 is transmitting data from three sensors at high speed.

4.5 Sampling Rate

```
const float SAMPLE_RATE_HZ = 800.0;
const uint32_t SAMPLE_PERIOD_US = (uint32_t)(1000000.0 / SAMPLE_RATE_HZ);
```

4.5.1 Purpose

These constants define the target sampling rate.

The sampling frequency is:

$$f_s = 800 \text{ Hz}$$

The sample period is:

$$T_s = \frac{1}{f_s}$$

$$T_s = \frac{1}{800}$$

$$T_s = 0.00125 \text{ s}$$

Since the ESP32 uses microseconds:

$$T_s = 1250 \mu s$$

The code calculates this as:

$$\text{SAMPLE_PERIOD_US} = \frac{1,000,000}{800}$$

$$\text{SAMPLE_PERIOD_US} = 1250$$

4.6 Number of Sensors

```
const int NUM_SENSORS = 3;
```

4.6.1 Purpose

This defines the number of MPU6050 sensors used by the system.

$$N_{\text{sensors}} = 3$$

4.7 Sensor Channels

```
const uint8_t SENSOR_CHANNELS[NUM_SENSORS] = {0, 1, 2};
```

4.7.1 Purpose

This array defines which PCA9548A channels are used.

Sensor 1 $\rightarrow$ Channel 0

Sensor 2 $\rightarrow$ Channel 1

Sensor 3 $\rightarrow$ Channel 2

The array is used whenever the firmware loops through the sensors.

4.8 Calibration Samples

```
const int CALIBRATION_SAMPLES = 1000;
```

4.8.1 Purpose

This defines how many samples are collected during calibration for each sensor.

$$N_{\text{calibration}} = 1000$$

The firmware averages these samples to estimate sensor offsets.

4.9 Accelerometer Scale Factor

```
const float ACCEL_SCALE = 4096.0;
```

4.9.1 Purpose

This constant represents the MPU6050 accelerometer sensitivity for the selected range.

The accelerometer is configured for:

$$\pm 8g$$

At this range:

$$4096 \text{ raw counts} = 1g$$

Although this constant is defined in the ESP32 script, the actual conversion to g is performed in the Python logger.

The conversion is:

$$a_g = \frac{a_{\text{raw}}}{4096}$$

4.10 Gyroscope Scale Factor

```
const float GYRO_SCALE = 131.0;
```

4.10.1 Purpose

This constant represents the MPU6050 gyroscope sensitivity for the selected range.

The gyroscope is configured for:

$$\pm 250^\circ/s$$

At this range:

$$131 \text{ raw counts} = 1^\circ/s$$

The Python logger converts raw gyroscope values using:

$$\omega_{\text{deg}/s} = \frac{\omega_{\text{raw}}}{131}$$

4.11 Binary Packet Constants

```
const uint8_t SYNC0 = 0xAA;  
const uint8_t SYNC1 = 0x55;  
const uint8_t PACKET_TYPE_DATA = 0x01;  
const uint8_t PAYLOAD_SIZE = 4 + 4 + (NUM_SENSORS * 6 * 2);
```

4.11.1 Purpose

These constants define the binary packet structure.

The two sync bytes are:

$0xAA$

and:

$0x55$

These help the Python logger find the beginning of each binary frame.

The packet type is:

$0x01$

This identifies the frame as a data packet.

The payload size is:

$$\text{Payload Size} = 4 + 4 + (3)(6)(2)$$

where:

- 4 bytes are used for timestamp.
- 4 bytes are used for sample index.
- 3 sensors are used.
- Each sensor sends 6 values.
- Each value is 2 bytes.

Therefore:

$$\text{Payload Size} = 4 + 4 + 36$$

$$\text{Payload Size} = 44 \text{ bytes}$$

The full frame size is:

$$2 + 1 + 1 + 44 + 1$$

= 49 bytes

The binary frame layout is:

sync0, sync1, packet type, payload size, payload, checksum

5 Data Structures and Global Variables

5.1 SensorOffset Structure

```
struct SensorOffset {  
    float ax = 0;  
    float ay = 0;  
    float az = 0;  
    float gx = 0;  
    float gy = 0;  
    float gz = 0;  
};
```

5.1.1 Purpose

This custom structure stores the calibration offset for one sensor.

It contains six fields:

- **ax**: acceleration offset in the x-axis
- **ay**: acceleration offset in the y-axis
- **az**: acceleration offset in the z-axis
- **gx**: gyroscope offset in the x-axis
- **gy**: gyroscope offset in the y-axis
- **gz**: gyroscope offset in the z-axis

5.1.2 Why Floating-Point Values Are Used

Offsets are calculated as averages. Averages may not be whole numbers, so they are stored as `float` values.

5.2 streamStartUs

```
uint32_t streamStartUs = 0;
```

5.2.1 Purpose

This stores the ESP32 timestamp at the moment binary streaming starts.

Each transmitted sample uses:

$$t_{\mu s} = \text{micros}() - \text{streamStartUs}$$

This makes the recorded time start near zero instead of using the absolute ESP32 uptime.

5.3 offsets

```
SensorOffset offsets[NUM_SENSORS];
```

5.3.1 Purpose

This array stores one `SensorOffset` structure for each sensor.

Since there are three sensors:

`offsets[0]`

`offsets[1]`

`offsets[2]`

5.4 sampleIndex

```
uint32_t sampleIndex = 0;
```

5.4.1 Purpose

This variable counts how many samples have been sent.

Each time a binary frame is created, the current sample index is written into the packet and then incremented.

The Python analyzer later uses this value to check for possible dropped samples.

6 Function Breakdown

6.1 selectPCAChannel()

```

void selectPCAChannel(uint8_t channel) {
    if (channel > 7) return;

    Wire.beginTransmission(PCA_ADDR);
    Wire.write(1 << channel);
    Wire.endTransmission(true);
}

```

6.1.1 Function Origin

This is a custom function written specifically for this project.

It does not come from a library.

6.1.2 Purpose

This function selects one active channel on the PCA9548A multiplexer.

Since all MPU6050 sensors use the same address, the ESP32 must select one multiplexer channel at a time.

6.1.3 Input

- **channel**: an unsigned 8-bit integer representing the PCA9548A channel number.

Valid values are:

$$0 \leq \text{channel} \leq 7$$

6.1.4 Output

This function does not return a value.

It changes the active I2C path on the PCA9548A.

6.1.5 Internal Library Functions Used

`Wire.beginTransmission()` Origin:

`Wire.h`

Purpose:

Starts an I2C transmission to the PCA9548A.

Input:

`PCA_ADDR`

Output:

No direct returned value.

`Wire.write()` Origin:

`Wire.h`

Purpose:

Writes the channel selection byte to the PCA9548A.

The expression:

`1 << channel`

creates a byte where only the selected channel bit is set.

Examples:

`1 << 0 = 00000001`

`1 << 1 = 00000010`

`1 << 2 = 00000100`

`Wire.endTransmission()` Origin:

`Wire.h`

Purpose:

Ends the I2C transmission.

The argument:

`true`

means that the I2C bus sends a stop condition.

6.1.6 How It Is Used in the Program

This function is called before communicating with any MPU6050 sensor.

Example:

```
selectPCAChannel(SENSOR_CHANNELS[sensor]);
```

This ensures that the correct sensor is connected to the ESP32 before reading or configuring it.

6.2 writeMPU()

```
void writeMPU(uint8_t reg, uint8_t value) {  
    Wire.beginTransaction(MPU_ADDR);  
    Wire.write(reg);  
    Wire.write(value);  
    Wire.endTransmission(true);  
}
```

6.2.1 Function Origin

This is a custom function written specifically for this project.
It does not come directly from a library.

6.2.2 Purpose

This function writes a value to a specific MPU6050 register.
It is used to configure the MPU6050.

6.2.3 Inputs

- `reg`: the MPU6050 register address.
- `value`: the value to write into that register.

6.2.4 Output

No return value.
The function sends a register write command over I2C.

6.2.5 Internal Library Functions Used

`Wire.beginTransaction()` Origin:

`Wire.h`

Purpose:
Starts communication with the MPU6050.

`Wire.write(reg)` Origin:

`Wire.h`

Purpose:
Sends the register address.

`Wire.write(value)` Origin:

`Wire.h`

Purpose:

Sends the value to be written into the register.

`Wire.endTransmission(true)` Origin:

`Wire.h`

Purpose:

Completes the I2C write operation.

6.2.6 How It Is Used

This function is used inside:

`setupMPUOnSelectedChannel()`

For example:

```
writeMPU(0x6B, 0x00);
```

This wakes up the MPU6050.

6.3 readMPU()

```
bool readMPU(int16_t &ax, int16_t &ay, int16_t &az,  
             int16_t &gx, int16_t &gy, int16_t &gz)
```

6.3.1 Function Origin

This is a custom function written specifically for this project.

It does not come from a library.

6.3.2 Purpose

This function reads acceleration and gyroscope data from the currently selected MPU6050 sensor.

It reads:

$a_x, a_y, a_z, g_x, g_y, g_z$

6.3.3 Inputs

The function receives six variables by reference:

- `ax`
- `ay`
- `az`
- `gx`
- `gy`
- `gz`

Because they are passed by reference, the function modifies the original variables passed into it.

6.3.4 Output

The function returns a Boolean value:

`true`

if the read was successful.

`false`

if communication failed.

It also outputs the six sensor values by updating the reference variables.

6.3.5 Internal Library Functions Used

`Wire.beginTransmission()` Origin:

`Wire.h`

Purpose:

Starts communication with the MPU6050.

`Wire.write(0x3B)` Origin:

`Wire.h`

Purpose:

Sets the MPU6050 register pointer to address:

`0x3B`

This is the first accelerometer data register.

`Wire.endTransmission(false)` Origin:

`Wire.h`

Purpose:

Ends the register-address write but keeps the I2C connection active using a repeated start.

The argument:

`false`

means no stop condition is sent.

This is useful because the ESP32 is immediately going to request data from that register.

`Wire.requestFrom()` Origin:

`Wire.h`

Purpose:

Requests bytes from the MPU6050.

In the script:

```
int bytesRead = Wire.requestFrom((int)MPU_ADDR, 14, (int>true);
```

This requests:

14 bytes

from the MPU6050.

`Wire.read()` Origin:

`Wire.h`

Purpose:

Reads one byte from the I2C receive buffer.

6.3.6 Why 14 Bytes Are Requested

The MPU6050 data block beginning at register 0x3B contains:

$$a_x, a_y, a_z, T, g_x, g_y, g_z$$

Each value is 16 bits:

$$16 \text{ bits} = 2 \text{ bytes}$$

There are 7 values:

$$7 \times 2 = 14 \text{ bytes}$$

The temperature value is read but ignored.

6.3.7 How the 16-bit Values Are Reconstructed

Each sensor value is transmitted as:

high byte, low byte

The code reconstructs each signed 16-bit value using:

$\text{value} = (\text{high byte} \ll 8) | (\text{low byte})$

For example:

```
ax = (int16_t)((Wire.read() << 8) | Wire.read());
```

This creates the signed 16-bit raw acceleration value for the x-axis.

6.3.8 How It Is Used

This function is used during:

- Sensor calibration.
- Binary frame generation.

6.4 mpuPresentOnSelectedChannel()

```
bool mpuPresentOnSelectedChannel() {  
    Wire.beginTransmission(MPU_ADDR);  
    return Wire.endTransmission(true) == 0;  
}
```

6.4.1 Function Origin

This is a custom function written specifically for this project.

6.4.2 Purpose

This function checks whether an MPU6050 sensor responds on the currently selected PCA9548A channel.

6.4.3 Input

No direct input.

The selected PCA9548A channel determines which sensor is being checked.

6.4.4 Output

Returns:

`true`

if the MPU6050 responds successfully.

Returns:

`false`

if no device responds at address:

`0x68`

6.4.5 Internal Library Functions Used

`Wire.beginTransmission()` Origin:

`Wire.h`

Purpose:

Starts communication with the MPU6050 address.

`Wire.endTransmission()` Origin:

`Wire.h`

Purpose:

Completes the I2C transaction and returns a status code.

If the return value is:

`0`

the transmission was successful.

6.4.6 How It Is Used

This function is used in `setup()` to check every configured sensor channel.

6.5 `setupMPUOnSelectedChannel()`

```
void setupMPUOnSelectedChannel() {  
    writeMPU(0x6B, 0x00);  
    delay(100);  
  
    writeMPU(0x1A, 0x01);  
    writeMPU(0x1B, 0x00);  
    writeMPU(0x1C, 0x10);  
}
```

```
writeMPU(0x19, 0x00);  
}
```

6.5.1 Function Origin

This is a custom function written specifically for this project.

6.5.2 Purpose

This function configures the MPU6050 sensor on the currently selected PCA9548A channel.

6.5.3 Input

No direct input.

The function acts on whichever sensor is currently connected through the selected PCA9548A channel.

6.5.4 Output

No return value.

It writes configuration values into MPU6050 registers.

6.5.5 Internal Functions Used

`writeMPU()` Origin:

Custom project function.

Purpose:

Writes values to MPU6050 registers.

`delay()` Origin:

`Arduino.h`

Purpose:

Pauses program execution.

In this function:

`delay(100)`

waits 100 milliseconds after waking the sensor.

6.5.6 Register Configuration

Register 0x6B: Power Management

```
writeMPU(0x6B, 0x00);
```

Purpose:

Wakes the MPU6050 from sleep mode.

Register 0x1A: Digital Low-Pass Filter

```
writeMPU(0x1A, 0x01);
```

Purpose:

Configures the digital low-pass filter.

The code comment notes that this gives approximately 44 Hz accelerometer bandwidth.

Register 0x1B: Gyroscope Range

```
writeMPU(0x1B, 0x00);
```

Purpose:

Sets the gyroscope full-scale range to:

$$\pm 250^{\circ}/s$$

Register 0x1C: Accelerometer Range

```
writeMPU(0x1C, 0x10);
```

Purpose:

Sets the accelerometer full-scale range to:

$$\pm 8g$$

Register 0x19: Sample Rate Divider

```
writeMPU(0x19, 0x00);
```

Purpose:

Sets the MPU6050 sample rate divider.

A value of zero means the sensor itself is not being intentionally slowed down by the divider.

6.6 clampToInt16()

```
int16_t clampToInt16(float value) {  
    if (value > 32767.0) return 32767;  
    if (value < -32768.0) return -32768;  
    return (int16_t)lround(value);  
}
```

6.6.1 Function Origin

This is a custom function written specifically for this project.

6.6.2 Purpose

This function ensures that a floating-point value can safely be stored as a signed 16-bit integer.

6.6.3 Input

- **value:** a floating-point value.

6.6.4 Output

Returns:

`int16_t`

The valid range of `int16_t` is:

$$-32768 \leq x \leq 32767$$

6.6.5 Internal Library Function Used

`lround()` Origin:

Available through the Arduino/C++ standard math support included by the Arduino framework.

Purpose:

Rounds a floating-point number to the nearest integer.

6.6.6 Why This Function Is Needed

During calibration correction, the code subtracts floating-point offsets from raw integer sensor data:

$$a_{\text{corrected}} = a_{\text{raw}} - a_{\text{offset}}$$

This result is a floating-point number.

However, the binary packet sends sensor values as signed 16-bit integers, so the value must be converted safely.

If the value is too high, it is limited to:

32767

If the value is too low, it is limited to:

-32768

6.7 `appendU8()`

```
void appendU8(uint8_t *buffer, uint8_t &index, uint8_t value) {  
    buffer[index++] = value;  
}
```

6.7.1 Function Origin

This is a custom helper function written specifically for this project.

6.7.2 Purpose

This function appends one unsigned 8-bit value to the binary frame buffer.

6.7.3 Inputs

- **buffer**: pointer to the binary frame array.
- **index**: current write position in the buffer, passed by reference.
- **value**: unsigned 8-bit value to append.

6.7.4 Output

No return value.

However, it modifies:

- The content of **buffer**.
- The value of **index**.

6.7.5 How It Works

The value is stored at the current buffer position:

```
buffer[index] = value
```

Then the index is incremented:

```
index = index + 1
```

6.7.6 Library Functions Used

No external library functions are used inside this function.

6.8 appendU32()

```
void appendU32(uint8_t *buffer, uint8_t &index, uint32_t value) {  
    buffer[index++] = (uint8_t)(value & 0xFF);  
    buffer[index++] = (uint8_t)((value >> 8) & 0xFF);  
    buffer[index++] = (uint8_t)((value >> 16) & 0xFF);  
    buffer[index++] = (uint8_t)((value >> 24) & 0xFF);  
}
```

6.8.1 Function Origin

This is a custom helper function written specifically for this project.

6.8.2 Purpose

This function appends a 32-bit unsigned integer to the binary frame buffer using little-endian byte order.

6.8.3 Inputs

- **buffer**: pointer to the binary frame array.
- **index**: current write position in the buffer, passed by reference.
- **value**: unsigned 32-bit integer to append.

6.8.4 Output

No return value.

It writes four bytes into the buffer and advances the index by four.

6.8.5 Little-Endian Format

Little-endian means the least significant byte is stored first.

For a 32-bit value:

$$\text{value} = b_3b_2b_1b_0$$

the function stores:

$$b_0, b_1, b_2, b_3$$

This matches how the Python logger later decodes the values with little-endian format.

6.8.6 Bitwise Operations Used

The expression:

$$\text{value} \& 0xFF$$

extracts the lowest 8 bits.

The expression:

$$\text{value} \gg 8$$

shifts the value right by 8 bits.

This allows the function to extract each byte of the 32-bit integer.

6.8.7 Library Functions Used

No external library functions are used inside this function.

6.9 appendI16()

```
void appendI16(uint8_t *buffer, uint8_t &index, int16_t value) {
    buffer[index++] = (uint8_t)(value & 0xFF);
    buffer[index++] = (uint8_t)((value >> 8) & 0xFF);
}
```

6.9.1 Function Origin

This is a custom helper function written specifically for this project.

6.9.2 Purpose

This function appends a signed 16-bit integer to the binary frame buffer using little-endian byte order.

6.9.3 Inputs

- **buffer**: pointer to the binary frame array.
- **index**: current write position in the buffer.
- **value**: signed 16-bit integer to append.

6.9.4 Output

No return value.

It writes two bytes into the buffer and advances the index by two.

6.9.5 Little-Endian Format

For a 16-bit value:

$$\text{value} = b_1b_0$$

the function stores:

$$b_0, b_1$$

The Python logger expects this same little-endian order.

6.9.6 Library Functions Used

No external library functions are used inside this function.

6.10 calculateChecksum()

```
uint8_t calculateChecksum(const uint8_t *buffer, uint8_t length) {
    uint8_t checksum = 0;

    for (uint8_t i = 0; i < length; i++) {
        checksum ^= buffer[i];
    }

    return checksum;
}
```

6.10.1 Function Origin

This is a custom function written specifically for this project.

6.10.2 Purpose

This function calculates an XOR checksum for the binary frame.

The checksum helps the Python logger detect corrupted frames.

6.10.3 Inputs

- **buffer**: pointer to the binary frame.
- **length**: number of bytes to include in the checksum.

6.10.4 Output

Returns an unsigned 8-bit checksum:

uint8_t

6.10.5 Checksum Equation

The checksum is calculated as:

$$C = b_0 \oplus b_1 \oplus b_2 \oplus \cdots \oplus b_n$$

where:

$\oplus$

represents bitwise XOR.

6.10.6 How It Is Used

The ESP32 calculates the checksum before transmitting the frame:

```
frame[idx++] = calculateChecksum(frame, idx);
```

The Python logger repeats the same calculation. If the expected checksum and received checksum do not match, the logger rejects the frame.

6.10.7 Library Functions Used

No external library functions are used inside this function.

6.11 calibrateSensors()

```
void calibrateSensors()
```

6.11.1 Function Origin

This is a custom function written specifically for this project.

6.11.2 Purpose

This function calculates sensor offsets for all three MPU6050 sensors.

The goal is to reduce static bias in the acceleration and gyroscope readings.

6.11.3 Inputs

No direct function inputs.

The function uses:

- NUM_SENSORS
- SENSOR_CHANNELS
- CALIBRATION_SAMPLES
- readMPU()
- selectPCAChannel()

6.11.4 Outputs

No return value.

However, it updates the global array:

`offsets`

Each sensor receives offset values for:

$a_x, a_y, a_z, g_x, g_y, g_z$

6.11.5 Internal Custom Functions Used

`selectPCAChannel()` Origin:

Custom project function.

Purpose:

Selects the PCA9548A channel for the sensor being calibrated.

`readMPU()` Origin:

Custom project function.

Purpose:

Reads raw accelerometer and gyroscope data from the selected sensor.

6.11.6 Internal Library Functions Used

`Serial.println()` Origin:

`Arduino.h`

Purpose:

Prints calibration status messages to the serial monitor.

`Serial.print()` Origin:

`Arduino.h`

Purpose:

Prints partial text messages to the serial monitor.

`delay()` Origin:

`Arduino.h`

Purpose:

Pauses execution.

In the calibration loop:

`delay(2)`

adds a 2 ms delay between calibration samples.

6.11.7 Calibration Process

For each sensor, the function:

1. Selects the sensor channel.
2. Initializes sum variables.
3. Reads 1000 samples.
4. Adds each valid reading to a running sum.
5. Counts the number of valid samples.
6. Divides each sum by the number of valid samples.
7. Stores the result in the `offsets` array.

6.11.8 Offset Equations

For the x-axis acceleration:

$$a_{x,\text{offset}} = \frac{1}{N} \sum_{i=1}^N a_{x,i}$$

For the z-axis acceleration:

$$a_{z,\text{offset}} = \frac{1}{N} \sum_{i=1}^N a_{z,i}$$

For the gyroscope x-axis:

$$g_{x,\text{offset}} = \frac{1}{N} \sum_{i=1}^N g_{x,i}$$

The same process is used for:

$$g_y, g_z$$

6.11.9 Special Treatment of the Y-Axis

The code uses:

```
aySum += ay - 4096;
```

This assumes that the sensor y-axis is vertical during calibration.
Since the accelerometer measures gravity, the vertical axis should measure approximately:

$$1g$$

At the selected $\pm 8g$ range:

$$1g = 4096 \text{ counts}$$

Therefore, subtracting 4096 removes the gravitational component from the y-axis calibration.

The y-axis offset equation is:

$$a_{y,\text{offset}} = \frac{1}{N} \sum_{i=1}^N (a_{y,i} - 4096)$$

6.11.10 Important Assumption

This calibration assumes the sensors are still during calibration.

If the printer or structure moves during calibration, the calculated offsets may be incorrect.

6.12 sendBinaryFrame()

```
void sendBinaryFrame()
```

6.12.1 Function Origin

This is a custom function written specifically for this project.

6.12.2 Purpose

This function creates and sends one complete binary data frame.

Each frame contains:

- Timestamp
- Sample index
- Sensor 1 data
- Sensor 2 data
- Sensor 3 data
- Checksum

6.12.3 Inputs

No direct function inputs.

The function uses:

- `micros()`
- `streamStartUs`
- `sampleIndex`
- `offsets`
- `SENSOR_CHANNELS`
- `readMPU()`
- `appendU8()`
- `appendU32()`
- `appendI16()`
- `calculateChecksum()`

6.12.4 Output

The output is one binary frame transmitted through:

`Serial.write()`

6.12.5 Internal Custom Functions Used

`appendU8()` Appends one 8-bit value to the frame.

`appendU32()` Appends one 32-bit value to the frame.

`appendI16()` Appends one signed 16-bit value to the frame.

`selectPCAChannel()` Selects each sensor through the PCA9548A.

`readMPU()` Reads sensor values.

`clampToInt16()` Converts corrected floating-point values back into safe signed 16-bit integers.

`calculateChecksum()` Calculates the final checksum byte.

6.12.6 Internal Library Functions Used

`micros()` Origin:

`Arduino.h`

Purpose:

Gets the current ESP32 time in microseconds.

`Serial.write()` Origin:

`Arduino.h`

Purpose:

Sends the binary frame through serial communication.

6.12.7 Timestamp Calculation

The frame timestamp is:

$$t_{\mu s} = \text{micros}() - \text{streamStartUs}$$

This makes the timestamp relative to the start of binary streaming.

6.12.8 Sample Index

The function appends:

`sampleIndex++`

This writes the current sample number and then increments it.

The sample index allows the Python analyzer to detect dropped samples.

6.12.9 Sensor Reading and Offset Correction

For each sensor, the code performs:

$$a_{x,\text{corrected}} = a_{x,\text{raw}} - a_{x,\text{offset}}$$

$$a_{y,\text{corrected}} = a_{y,\text{raw}} - a_{y,\text{offset}}$$

$$a_{z,\text{corrected}} = a_{z,\text{raw}} - a_{z,\text{offset}}$$

$$g_{x,\text{corrected}} = g_{x,\text{raw}} - g_{x,\text{offset}}$$

$$g_{y,\text{corrected}} = g_{y,\text{raw}} - g_{y,\text{offset}}$$

$$g_{z,\text{corrected}} = g_{z,\text{raw}} - g_{z,\text{offset}}$$

The corrected values are then clamped to signed 16-bit integer range.

6.12.10 Failure Handling

If a sensor read fails, the code sends zeros for that sensor.

This prevents the binary frame structure from breaking.

6.12.11 Binary Frame Layout

The frame structure is:

Field	Size	Description
SYNC0	1 byte	First sync byte, 0xAA
SYNC1	1 byte	Second sync byte, 0x55
Packet type	1 byte	Data packet type, 0x01
Payload size	1 byte	Number of payload bytes
Timestamp	4 bytes	Time since stream start in microseconds
Sample index	4 bytes	Sequential sample number
Sensor 1 data	12 bytes	ax, ay, az, gx, gy, gz
Sensor 2 data	12 bytes	ax, ay, az, gx, gy, gz
Sensor 3 data	12 bytes	ax, ay, az, gx, gy, gz
Checksum	1 byte	XOR checksum

The total frame size is:

$$2 + 1 + 1 + 44 + 1 = 49 \text{ bytes}$$

6.13 `setup()`

```
void setup()
```

6.13.1 Function Origin

`setup()` is a standard Arduino framework function.

It comes from the Arduino program structure, not from the user-defined project code.

However, the contents inside `setup()` are custom-written for this project.

6.13.2 Purpose

The `setup()` function runs once when the ESP32 starts.

It initializes the full data acquisition system.

6.13.3 Inputs

No direct inputs.

6.13.4 Outputs

No return value.

It initializes hardware and prepares the system for streaming.

6.13.5 Internal Library Functions Used

`Serial.begin()` Origin:

`Arduino.h`

Purpose:

Starts serial communication.

`delay()` Origin:

`Arduino.h`

Purpose:

Pauses the program for a defined number of milliseconds.

`Wire.begin()` Origin:

`Wire.h`

Purpose:

Starts I2C communication on the selected SDA and SCL pins.

`Wire.setClock()` Origin:

`Wire.h`

Purpose:

Sets the I2C clock speed.

The code uses:

800000 Hz

or:

800 kHz

`Serial.print()` and `Serial.println()` Origin:

`Arduino.h`

Purpose:

Print startup and status messages.

`micros()` Origin:

`Arduino.h`

Purpose:

Stores the start time for the binary stream.

`Serial.flush()` Origin:

`Arduino.h`

Purpose:

Waits for outgoing serial data to finish transmitting.

6.13.6 Internal Custom Functions Used

- `selectPCAChannel()`
- `mpuPresentOnSelectedChannel()`
- `setupMPUOnSelectedChannel()`
- `calibrateSensors()`

6.13.7 Setup Sequence

The setup sequence is:

1. Start serial communication.
2. Wait 1 second.
3. Start I2C communication on pins 21 and 22.
4. Set I2C clock to 800 kHz.
5. Print startup information.
6. Loop through all three sensors.
7. Select each PCA9548A channel.
8. Check if an MPU6050 is present.

9. Configure each found MPU6050.
10. Calibrate all sensors.
11. Reset the sample index.
12. Store the binary stream start time.
13. Print `START_BINARY`.
14. Flush the serial output.

6.13.8 Importance of `START_BINARY`

The line:

```
Serial.println("START_BINARY");
```

is very important.

The Python logger waits for this exact text before it begins reading binary frames.

This prevents the logger from accidentally interpreting startup text as binary sensor data.

6.14 `loop()`

```
void loop() {
    static uint32_t lastSample = micros();
    uint32_t now = micros();

    if ((uint32_t)(now - lastSample) >= SAMPLE_PERIOD_US) {
        lastSample += SAMPLE_PERIOD_US;
        sendBinaryFrame();
    }
}
```

6.14.1 Function Origin

`loop()` is a standard Arduino framework function.

It is part of the Arduino program structure.

However, the code inside it is custom-written for this project.

6.14.2 Purpose

The `loop()` function controls the sampling rate and repeatedly sends sensor frames.

6.14.3 Inputs

No direct inputs.

It uses:

- `micros()`
- `SAMPLE_PERIOD_US`
- `sendBinaryFrame()`

6.14.4 Outputs

No direct return value.

It sends binary frames indirectly by calling:

`sendBinaryFrame()`

6.14.5 Internal Library Functions Used

`micros()` Origin:

`Arduino.h`

Purpose:

Returns current time in microseconds.

6.14.6 Internal Custom Functions Used

`sendBinaryFrame()` Origin:

Custom project function.

Purpose:

Reads the sensors and sends one binary frame.

6.14.7 Sampling Rate Logic

The loop compares:

`now - lastSample`

against:

`SAMPLE_PERIOD_US`

If enough time has passed, one new frame is sent.

The condition is:

$\text{now} - \text{lastSample} \geq \text{SAMPLE_PERIOD_US}$

For this script:

$$\text{SAMPLE_PERIOD_US} = 1250$$

Therefore, one sample is sent approximately every:

$$1250 \mu s$$

which corresponds to:

$$800 \text{ Hz}$$

6.14.8 Why `lastSample += SAMPLE_PERIOD_US` Is Used

The code uses:

```
lastSample += SAMPLE_PERIOD_US;
```

instead of:

```
lastSample = now;
```

This is important because it keeps the sampling schedule locked to an ideal time grid. If `lastSample = now` were used, small delays would accumulate and create timing drift. Using:

$$\text{lastSample} = \text{lastSample} + T_s$$

helps maintain a more consistent sampling period.

7 How the ESP32 Reads Sensor Data

The ESP32 reads one sensor at a time.

For each sensor:

1. The ESP32 selects the PCA9548A channel.
2. The ESP32 tells the MPU6050 to begin reading at register 0x3B.
3. The ESP32 requests 14 bytes.
4. The ESP32 reconstructs six useful signed 16-bit values.
5. The temperature bytes are ignored.

The useful values are:

$$a_x, a_y, a_z, g_x, g_y, g_z$$

The temperature value is not used in the vibration analysis.

8 How the ESP32 Outputs Data

The ESP32 sends binary data rather than text data.

8.1 Why Binary Output Is Used

Binary output is more efficient than text output.

For example, the integer:

12345

as text requires five ASCII characters.

As binary, a 16-bit integer requires only:

2 bytes

Since the project sends high-speed data from three sensors, binary output helps reduce the amount of serial data.

8.2 Full Frame Size

Each frame is:

49 bytes

At a target rate of:

800 frames/s

the binary data rate is:

$$49 \times 800 = 39200 \text{ bytes/s}$$

In bits per second:

$$39200 \times 8 = 313600 \text{ bits/s}$$

The serial baud rate is:

$$921600 \text{ bits/s}$$

Therefore, the selected baud rate provides enough theoretical bandwidth for this packet size.

9 Sampling Rate Implementation

The firmware controls the target sampling rate in software using the ESP32 microsecond timer.

The target frequency is:

$$f_s = 800 \text{ Hz}$$

The sampling period is:

$$T_s = \frac{1}{800}$$

$$T_s = 0.00125 \text{ s}$$

$$T_s = 1250 \text{ } \mu\text{s}$$

The firmware sends a frame when the elapsed time reaches this period. However, the real sampling rate may be affected by:

- I2C communication time.
- Multiplexer switching time.
- Reading three sensors sequentially.
- Serial transmission time.
- ESP32 processing time.

For this reason, the Python analyzer later estimates the true sampling rate from the timestamps instead of assuming it is exactly 800 Hz.

10 Complete ESP32 Firmware Execution Flow

The complete firmware execution sequence is:

1. ESP32 starts.
2. `setup()` begins.
3. Serial communication starts at 921600 baud.
4. I2C communication starts on pins 21 and 22.
5. I2C clock is set to 800 kHz.
6. Sensor 1 channel is selected.

7. Sensor 1 is checked.
8. Sensor 1 is configured if present.
9. Sensor 2 channel is selected.
10. Sensor 2 is checked.
11. Sensor 2 is configured if present.
12. Sensor 3 channel is selected.
13. Sensor 3 is checked.
14. Sensor 3 is configured if present.
15. All sensors are calibrated.
16. Sample index is reset.
17. Stream start time is recorded.
18. ESP32 prints `START_BINARY`.
19. `loop()` begins running continuously.
20. The loop waits until 1250 μs have elapsed.
21. A binary frame is created.
22. Each sensor is read sequentially.
23. Corrected values are appended to the binary frame.
24. Checksum is calculated.
25. Frame is transmitted through serial.
26. Process repeats continuously.

11 Important Design Decisions

11.1 Use of Binary Packets

Binary packets are used because they reduce the amount of data sent over serial.

This helps the ESP32 maintain a higher sampling rate.

11.2 Use of Sync Bytes

The sync bytes:

`0xAA, 0x55`

allow the Python logger to find the beginning of each frame.

This is important if the logger starts reading in the middle of a packet or if one packet becomes corrupted.

11.3 Use of Checksum

The checksum allows the Python logger to detect corrupted packets.

If a packet fails the checksum test, the logger discards it.

11.4 Use of Sample Index

The sample index allows the analyzer to check whether samples were skipped or repeated.

11.5 Use of Timestamp

The timestamp allows the analyzer to estimate the real sampling rate.

This is important because the real sampling rate may differ from the target sampling rate.

12 Summary

The ESP32 firmware is responsible for real-time data acquisition.

It performs:

- I2C communication
- Multiplexer channel selection
- MPU6050 configuration
- Sensor calibration
- Multi-sensor reading
- Binary packet construction
- Checksum calculation
- Serial transmission
- Sampling rate control

The most important custom functions are:

- `selectPCAChannel()`
- `writeMPU()`
- `readMPU()`
- `mpuPresentOnSelectedChannel()`
- `setupMPUOnSelectedChannel()`
- `clampToInt16()`
- `appendU8()`
- `appendU32()`
- `appendI16()`
- `calculateChecksum()`
- `calibrateSensors()`
- `sendBinaryFrame()`

The most important library functions are:

- `Wire.begin()`
- `Wire.setClock()`
- `Wire.beginTransmission()`
- `Wire.write()`
- `Wire.endTransmission()`
- `Wire.requestFrom()`
- `Wire.read()`
- `Serial.begin()`
- `Serial.print()`
- `Serial.println()`
- `Serial.write()`
- `Serial.flush()`
- `delay()`
- `micros()`

Overall, this firmware transforms raw sensor measurements into a structured binary data stream that can be decoded and analyzed by the Python scripts.